

Drought Risk Score Forecasting Using LightGBM Multi-Output Regression with Seasonal Feature Engineering

Dhammaruci Wiradhika Lungadhi
Department of Electrical Engineering
and Computer Science
National Yang-Ming Chiao-Tung
University
Hsinchu, Taiwan
wiradhikalungadhi@gmail.com

Yen Yun, Chen
Department of Computer Science
National Yang-Ming Chiao-Tung
University
Hsinchu, Taiwan
yychen.p@gmail.com

Aye Mya Thandar
Department of Global MBA
National Yang-Ming Chiao-Tung
University
Hsinchu, Taiwan
ayemyathandar.mg13@nycu.edu.tw

Abstract—This paper presents an iterative machine learning approach to weekly drought risk score forecasting across multiple geographic regions. The problem is formulated as a supervised multi-output regression task: given daily meteorological observations per region, simultaneously predict drought risk scores (range 0–5) for the following five consecutive weeks. Starting from a simple LightGBM baseline with basic rolling window features, the methodology was progressively refined through systematic exploration of ensemble blending, deep learning architectures (Transformer, CNN-GRU), velocity features, climate normal baselines, anomaly encoding, a composite evaporation stress index, a future-month seasonal embedding, and finally an evaporation demand ratio. The full development history spans two Kaggle notebooks comprising 52 cells (raw) and 15 cells (final), with over 50 Kaggle submissions.

Our best-performing model achieves a public leaderboard MAE of 0.7896, a 28.0% improvement over the initial weighted ensemble baseline (MAE 1.0952).

Group 12 Github:
https://github.com/twilight123winni/DM2026_Final_Project

README.md in the repository provides step-by-step instructions for reproducing all results on a Kaggle kernel environment with GPU acceleration.

Keywords—drought risk forecasting; LightGBM; gradient boosting; multi-output regression; time-series; feature engineering; climate normals; evaporation stress; evaporation demand; Transformer; CNN-GRU

I. INTRODUCTION

Drought is among the most economically devastating natural hazards, threatening agricultural production, water supply, and ecosystem stability across wide geographic areas. Accurate multi-week drought risk forecasts enable water utilities, agricultural planners, and emergency agencies to implement adaptive measures with meaningful lead time. This work addresses drought risk forecasting as a tabular time-series multi-output regression challenge on the Kaggle competition dataset provided for the Data Mining course (Spring 2026).

The task is to simultaneously predict a drought risk score (range 0–5) for each of five consecutive future weeks, given daily meteorological records per geographic region. This report documents the complete development journey across

two Kaggle notebooks: a raw exploration notebook (52 cells) and a streamlined final notebook (15 cells), together comprising more than 50 leaderboard submissions.

The development followed a deliberate experimental arc: beginning with simple LightGBM baselines evaluated on chronological train/validation splits, progressing through ensemble blending, deep learning architectures, and systematic feature engineering, before converging on a refined LightGBM multi-output pipeline. The best submission achieves MAE 0.7896, a 28.0% improvement over the initial ensemble baseline (MAE 1.0952). All experiments are reproducible from the GitHub repository; README.md provides step-by-step Kaggle Notebook execution instructions.

II. PROJECT SUMMARY

A. Goal

The objective of this project is to predict weekly drought scores for multiple geographic regions up to five weeks into the future. Each region has a series of daily weather measurements, and the task requires simultaneously forecasting five weekly targets using only information available training date.

B. Problem Formulation

Let $R = \{1, \dots, N\}$ denote the set of geographic regions and let $x(t) \in \mathbb{R}^d$ denote the feature vector of daily meteorological observations for region r at date t . The task is to learn a function F such that:

$$F(x_r(t)) = [s_r(t + 7), s_r(t + 14), s_r(t + 21), s_r(t + 28), s_r(t + 35)]$$

where $s_r(t + 7k)$ is the drought risk score for region r at date $t + 7k$ ($k = 1, \dots, 5$). The evaluation metric is Mean Absolute Error (MAE) averaged across all regions, horizons, and test rows:

```
train_raw[target_wk]=  
train_raw.groupby('region_id')['score'].shift  
(-(k)*7)
```

This formulation is equivalent to the DIRECT multi-step forecasting strategy, where each horizon k is treated as an independent regression target.

C. High-Level Method

The core of our approach is a gradient-boosted decision tree model (LightGBM) with multi-output regression. Rather than training five separate models or using autoregressive chains, we train a single model that outputs all five weekly

predictions simultaneously. This is achieved via scikit-learn's MultiOutputRegressor, which fits one LGBMRegressor per target, sharing the same feature set.

The key insight driving our feature engineering is that drought risk is climatologically seasonal: the same region will tend to have similar risk levels in the same month each year. We exploit this by computing climate normals (region-month averages) and encoding the expected future month for each prediction horizon, allowing the model to anticipate seasonal transitions.

III. RELATED WORK

Gradient-boosted decision trees (GBDT) dominate tabular regression benchmarks. Chen and Guestrin [1] introduced XGBoost with regularised boosting and column subsampling. Ke et al. [2] proposed LightGBM with leaf-wise growth and histogram-based splitting for efficient GPU training. Grinsztajn et al. [7] benchmarked GBDT against modern deep learning on 45 tabular datasets, showing consistent GBDT superiority—a finding confirmed by our own deep learning experiments.

For multi-step horizon forecasting, Hyndman and Athanasopoulos [3] describe the trade-offs between the RECURSIVE strategy (one model, predictions fed back) and the DIRECT strategy (one model per horizon). We implement DIRECT via scikit-learn's MultiOutputRegressor [4], fitting one LGBMRegressor per target week. Early experiments also tested RegressorChain [4], which chains predictions across horizons but is susceptible to error propagation.

McKee et al. [5] introduced the Standardized Precipitation Index (SPI), a widely used drought metric built on precipitation anomalies—conceptually related to our anomaly features. Mishra and Singh [6] review multi-scale temporal rolling windows as essential for capturing drought dynamics at different time scales. The Transformer [8] and CNN-RNN hybrids have been applied to meteorological time-series but consistently underperform GBDT on tabular data [7], as confirmed by our experiments.

IV. PROPOSED METHOD

A. Data Preprocessing

The training and test CSV files contain daily weather observations per region. Preprocessing steps include:

- Date parsing: extracting year, month, and day from the date string for use as temporal features.
- Sorting by region_id and date to ensure temporal ordering within each region's time series.
- Encoding region_id as a Pandas categorical type, which enables us to extract the historical and monthly average score for every region_id.
- Target construction via shifted groupby operations: $\text{target_w}\{k\} = \text{score shifted by } k*7 \text{ days within each region.}$

B. Phase 1: Initial Exploration and Baselines (Raw Notebook, Cells 0–13)

The raw notebook begins with Kaggle environment setup (Cell 0): importing NumPy and Pandas, and listing available input files via os.walk. Cell 1 performs the first exploratory

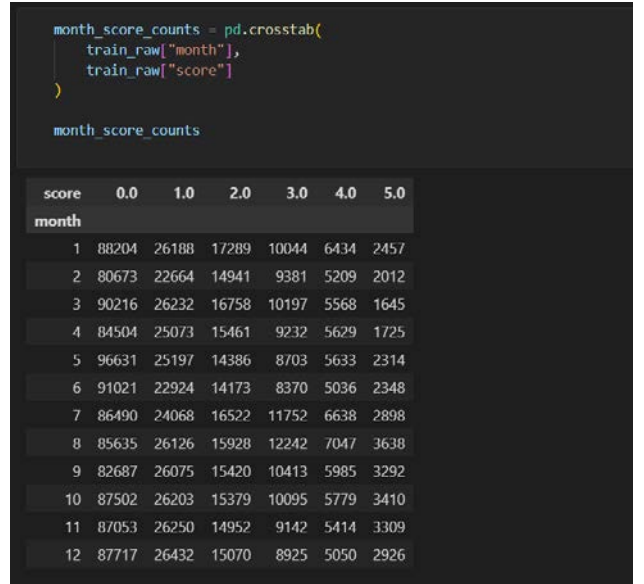


Fig. 1. Data distribution observation

data analysis (EDA): `df['score'].value_counts()` to understand the distribution of drought risk scores across the dataset.

Cell 2 establishes the first LightGBM model: a MultiOutputRegressor wrapping LGBMRegressor (objective='mae', n_estimators=100). Features are all available columns excluding identifiers and targets. The dataset is split chronologically at the 80th percentile date (80/20 train/validation). This produces the first validation MAE as a baseline reference. Cell 3 prints the top feature importances for Week 1.

Cell 4 adds the first engineered feature: `region_historical_avg_score`, computed as `df.groupby('region_id')['score'].mean()`. Region identity is encoded as a Pandas category dtype to enable LightGBM's native categorical splitting. The model is retrained with the same 80/20 split. Cell 5 experiments with five separate per-target specialized models, each receiving a manually curated feature set indexed by forecast horizon: short-range features (7- or 14-day windows) for `target_w1` and progressively longer features (up to 91-day windows) for `target_w5`.

Cell 6 implements TimeSeriesSplit (n_splits=3) as a rigorous chronological cross-validation framework. For each fold, three random dates are sampled from the validation period to simulate Kaggle submission cadence, and the fold-average MAE is computed across mock submissions. Cell 7 constructs a strict 91-day rolling window simulation: for 10 random cutoff dates, exactly 91 days of history are extracted per region, features are built on that isolated chunk (preventing data leakage), and the pre-trained model is evaluated. Weather features include rolling mean/max for `prec` and `tmp_max` at windows [7, 14, 35, 70, 91] days, plus score lags at [7, 14, 21, 28, 35] days.

Cell 8 constructs a region-centric mock test set by sampling 5 random valid scoring days per region from the validation period. Cell 9 implements a full self-contained pipeline: `weather_cols = [prec, tmp_max, humidity, wind_max, surf_pre]`, `windows = [7, 14, 35, 70, 91]`, score lags at [14, 21, 28, 35] days, chronological 80/20 split, `n_estimators=150`. Cells 11 and 13 represent the first end-to-end pipeline producing a real test-set submission: the model is

trained on 100% of training data, and the last row per region from the test set is used for inference.

C. Phase 2: Climate Normals, Ensemble Blending, and Feature Expansion (Raw Notebook, Cells 14–32)

Cell 14 introduces the first climate normal baseline: historical region-month averages of `prec` and `tmp_max` are computed from training data and merged into both train and test as `normal_prec` and `normal_tmp_max`. Global monthly fallbacks are also computed for unseen region-month combinations. A `RegressorChain` is tested as an alternative to `MultiOutputRegressor`, chaining predictions across horizons.

Cell 16 introduces `evap_stress = tmp_max × (100 – humidity)`, described in the notebook comment as “Atmospheric Thirst Feature.” Climate normals are expanded to cover five variables: [`prec`, `tmp_max`, `humidity`, `surf_pre`, `evap_stress`]. Cell 17 adds velocity features: `prec_velocity = prec_roll_mean_7 – prec_roll_mean_35` and `tmp_velocity = tmp_max_roll_mean_7 – tmp_max_roll_mean_35`, encoding the rate of change of recent weather conditions relative to the medium-term trend.

Cell 18 implements the “Tabular Holy Trinity” ensemble: `LightGBM` (`n_estimators=450`, `lr=0.03`, `num_leaves=63`), `XGBoost` (`n_estimators=400`, `lr=0.03`, `max_depth=7`), and `CatBoost` (`iterations=500`, `lr=0.04`, `depth=6`) blended at weights 0.50/0.25/0.25. Cell 19 produces the first real Kaggle submission: a `LightGBM` (74%) + `XGBoost` (26%) weighted blend using `evap_stress`, climate normals, and windows [7, 14, 28, 49, 84]. This submission (`submission_weighted.csv`) achieves MAE 1.0952. Cell 21 instantly re-blends the same predictions to 60/40 ratio without retraining.

Cells 22–24 explore `XGBoost` with `KFold` cross-validation (`n_splits=5`, `shuffle=True`), sample weighting via `compute_sample_weight` (`class_weight='balanced'`) to address score class imbalance, and windows [7, 14, 21, 28, 35, 70]. Cell 25 implements a `TabularNN` with architecture 128→64→32→16→1, one network per target week, using 5-fold `KFold` and `StandardScaler`. Cells 26, 28, and 30 are progressive `LightGBM` refinements: expanding the baseline weather column set from 5 to 8 variables and testing various window configurations. Cell 27 prints top-10 features per week.

D. Phase 2: Deep Learning Experiments (Raw Notebook, Cells 31–32)

Cell 31 implements a full Transformer model (`submission_nn_only_ram_friendly.csv`, MAE 1.1040). The architecture uses sequence length 90 days, batch size 64, 25 epochs, learning rate 1×10^{-3} , weight decay 1×10^{-4} , and patience 5 for early stopping. The Transformer processes the weather sequence using multi-head self-attention and outputs all five weekly predictions simultaneously. Feature columns include raw weather variables plus `evap_stress`, with rolling statistics at windows [7, 14, 28, 49, 84].

Cell 32 implements a hybrid CNN-GRU model. The architecture uses sequence length 90, batch size 128, 20 epochs, cosine annealing learning rate scheduler, and cyclical time encoding (`sin/month` and `cos/month` features). Static features (`month_region_score`, future horizon scores) are concatenated to dynamic sequence outputs. This model produces `submission_hybrid_cnn_gru.csv` (MAE 0.9787). Both deep learning architectures substantially underperformed `LightGBM`, consistent with [7].

E. Phase 3: Final LightGBM Pipeline — EDA and Baseline (Final Notebook, Cells 1–7 / Raw Cells 34–40)

Cell 33 of the raw notebook (Cell 0 of the final notebook) marks the clean restart with the comment “Starting From Here is the final one.” Cell 34/1 sets up the complete pipeline skeleton: imports, GPU device check (`torch.device('cuda')`), CSV loading from the Kaggle input path, date parsing via `str.split('-', expand=True)` into integer year/month/day fields, and `region_score_counts` EDA cross-tabulation.

Cells 35–38 (final Cells 3–6) perform systematic EDA: checking for zero-score regions, `month × score` cross-tabulation (`month_score_counts`), `day × score` cross-tabulation (`day_score_counts`), and the key (`region_id`, `month`) `× score` cross-tabulation (`region_monthly_score_counts`). Cell 39 generates a heatmap of region-month average scores using Seaborn. This EDA confirms strong regional heterogeneity and pronounced monthly seasonality, directly motivating the region-month feature engineering strategy.

Cell 40 (final Cell 7) implements the first clean final pipeline: `evap_stress` is computed; climate normals are built for `weather_cols = [prec, tmp, humidity, wind, surf_tmp, evap_stress]` over (`region_id`, `month`); targets are constructed via `groupby` shift; rolling windows = [7, 14, 28, 49, 84] computing mean and max (no std); anomaly features = `roll_mean – normal`; only historical `region_score` is added (no `month_region_score` yet, no future-month embedding). Model: `n_estimators=1000`, `num_leaves=95`, `lr=0.015`, `subsample=0.8`, `colsample_bytree=0.8`, `device='gpu'`. This produces `submission_70_window_region.csv` (MAE 0.8582).

F. Phase 4: Monthly Score Priors and Future-Month Embedding (Final Cell 8 / Raw Cell 41)

Cell 41 (final Cell 8) adds two critical features. First, `month_region_score` is computed as `groupby(['region_id', 'month'])['score'].mean()`, encoding the historical average drought risk for each (region, calendar month) pair. Second, the future-month seasonal embedding function `add_future_month_features` is introduced. For each horizon $h = 1, \dots, 5$, a calendar-aware helper function steps forward $h \times 7$ days from the current (year, month, day):

```
MONTH_LEN = {1:31, 2:28, 3:31, 4:30,
              5:31, 6:30, 7:31, 8:31, 9:30, 10:31, 11:30,
              12:31}

def add_days_get_future_month(year,
                              month, day, add_days):
    # steps forward day-by-day, wrapping
    month/year; handles Feb 29 via
    is_leap_year()

    return future_calendar_month #
    integer 1-12
```

The resulting future calendar month is used to look up `month_region_score` for (`region_id`, `future_month`), appended as feature `future_region_month_score_w{h}`. This provides a forward-looking seasonal prior for each forecast horizon. Rolling std features are now computed for $w \geq 28$ days. Adaptive rounding $\theta = 0.20$ is applied. This produces `submission_70_window_region_monthly.csv` (MAE 0.8526).

Cell 42 (final Cell 9) is a standalone rounding experiment: the dual-threshold scheme (floor at $\theta = 0.20$ to snap near-integer predictions downward, then round at $\theta = 0.10$) is applied to the existing model output, producing

submission_70_window_region_monthly_down_20_up_10.csv.

G. Phase 5: Expanded Weather Columns and 5-Week Windows (Final Cell 10 / Raw Cell 43)

Cell 43 (final Cell 10) makes two key changes. First, `weather_cols` is expanded from 6 to 8 variables: `[prec, humidity, wind, surf_tmp, evap_stress, surf_pre, dp_tmp, tmp]`. Second, the window set is changed from `[7, 14, 28, 49, 84]` to `[7, 14, 21, 28, 35]`—matching the five-week forecast horizon exactly and using uniform weekly increments. Roll std features are computed for $w \geq 35$ only. The `evap_demand` feature is not yet present. This produces `submission_last_5_week_window_region_monthly.csv` (MAE 0.8298).

H. Phase 6: Larger Model and Stronger Regularisation (Final Cell 11 / Raw Cells 44–47)

Cell 44 (final Cell 11) upgrades the model configuration substantially: `n_estimators` is increased from 1000 to 5000; `learning_rate` is reduced from 0.015 to 0.01; `num_leaves` is reduced from 95 to 63; `subsample` and `colsample_bytree` are reduced from 0.8 to 0.6; `subsample_freq = 1` is added; `lambda_l1 = 2` and `lambda_l2 = 20` are introduced for strong L1/L2 regularisation. Roll std features are now computed from $w \geq 28$ (not ≥ 35). The rounding threshold is changed to $\theta = 0.25$. This produces `submission_last_5_week_window_region_monthly.csv` (MAE 0.8085). Cells 45–47 of the raw notebook are the “Consider ACF and PACF” note and two reruns of the same configuration confirming the result.

I. Phase 7: Best-Performing Model — `evap_demand`, Extended Window, `min_periods=1` (Final Cell 12 / Raw Cell 49)

Cell 49 of the raw notebook (final Cell 12) implements the best-performing configuration. Three targeted changes are made to the Phase 6 model:

- `evap_demand` is added as a new composite drought index: `train_raw['evap_demand'] = train_raw['tmp_max'] / (train_raw['prec'] * 0.01)`. This captures the atmospheric moisture demand-to-supply ratio and is included in `weather_cols`, expanding it to 9 variables.
- Window set is changed from `[7, 14, 21, 28, 35]` to `[7, 14, 21, 28, 42]`, replacing the 5-week maximum with a 6-week (42-day) maximum to provide a slightly longer medium-range memory.
- `min_periods=1` is added to all rolling window calls: `x.rolling(w, min_periods=1)`. This allows statistics to be computed from the very first observation in each region's history rather than producing NaN for the initial $w-1$ rows, maximising usable training rows and ensuring complete test-set coverage.

The rounding threshold is set to $\theta = 0.20$. This model produces `submission_last_4&8_weeks_window_region_monthly.csv` (MAE 0.7896), which is the selected best submission. The file name contains a typographical error acknowledged in the notebook comment; the correct description is “windows covering 4- to 6-week scales.”

J. Phase 8: Trial Experiments (Final Cell 14 / Raw Cell 50)

Cell 50 of the raw notebook (final Cell 14) is placed after the “above is the latest, below is trial” marker and represents further experimental attempts not selected for final submission. Key differences from the best model: `n_estimators` is increased to 6000; `subsample` and `colsample_bytree` are increased to 0.7; the rounding scheme is changed to a two-stage approach: first snap values within $\theta = 0.10$ of an integer to the floor, then force any prediction below 0.25 to exactly 0.0 (an aggressive zero-snap heuristic intended to eliminate near-zero noise). This produces `submission_last_5_week_window_region_monthly_10_rounded.csv` (MAE 0.8159), which is worse than the selected submission, confirming that the zero-snap heuristic is counterproductive.

V. FINAL METHOD SUMMARY

The selected best submission (MAE 0.7896) corresponds to final notebook Cell 12. This section provides a complete specification of the final method for reproducibility.

A. Feature Engineering Pipeline

All features are derived exclusively from the raw dataset columns without external data.

(1) Composite Drought Indices

```
evap_stress = tmp_max * (100 - humidity)
# Atmospheric Thirst

evap_demand = tmp_max / (prec * 0.01)
# Moisture Demand Ratio
```

`evap_stress` captures combined heat and atmospheric dryness. `evap_demand` encodes the ratio of energy input to precipitation supply. Both are included in all subsequent rolling and anomaly computations.

(2) Climate Normal Baselines

```
weather_cols = [prec, humidity, wind,
                 surf_tmp, evap_stress, surf_pre, dp_tmp,
                 tmp, evap_demand]

normals = train.groupby(['region_id', 'month'])[weather_cols].mean()

# merged into both train and test as normal_{col}
```

(3) Rolling Window Statistics

```
windows = [7, 14, 21, 28, 42]

# roll_mean, roll_max for all windows;
# roll_std for w >= 28

# all with min_periods=1
```

(4) Anomaly Features

```
anomaly_{col}_{w} = roll_mean_{col}_{w} - normal_{col}
```

(5) Region Score Priors

```
historical_region_score = groupby('region_id')['score'].mean()

month_region_score = groupby(['region_id', 'month'])['score'].mean()
```

(6) Future-Month Seasonal Embedding

```
future_region_month_score_w{h} = month_region_score[region, future_month(t + h*7)]
```

```
# computed via calendar-aware
add_days_get_future_month() for h=1..5
```

B. Model Configuration

TABLE I. Final Model Hyperparameters (Cell 12 / Best Submission)

PARAMETER	VALUE
objective	mae
n_estimators	5000
num_leaves	63
learning_rate	0.01
subsample	0.6
subsample_freq	1
colsample_bytree	0.6
lambda_l1	2
lambda_l2	20
device	gpu
random_state	42

C. Adaptive Rounding Post-Processing

```
rounded = floor(pred)
distance = |pred - rounded|
output = rounded if distance < 0.20
else pred
```

Predictions are first clipped to $[0, 5]$, then the adaptive rounding rule with $\theta = 0.20$ is applied. This selectively snaps only predictions very close to an integer, preserving meaningful fractional values elsewhere.

VI. EXPERIMENTS

A. Dataset Description

The dataset contains daily meteorological observations across multiple geographic regions, each annotated with a drought risk score (0–5). Table II lists all raw features. EDA (final Cells 2–6) revealed strong regional heterogeneity in baseline drought risk (region cross-tabs) and pronounced monthly seasonality within regions (month cross-tabs and the region-month heatmap), directly motivating the region-month feature engineering strategy.

TABLE II. Dataset Features

FEATURE	TYPE	DESCRIPTION
region_id	Categorical	Geographic region identifier
date	String	Observation date (YYYY-MM-DD)
prec	Float	Daily precipitation
tmp	Float	Mean daily temperature
tmp_max	Float	Maximum daily temperature
humidity	Float	Relative humidity (%)
wind	Float	Wind speed
surf_tmp	Float	Surface temperature

surf_pre	Float	Surface pressure
dp_tmp	Float	Dew point temperature
score	Integer (0–5)	Daily drought risk score (prediction target)

B. Complete Submission History

Table III documents every key submission produced across both notebooks, mapping each to its notebook cell, the key change introduced, and the resulting public leaderboard MAE.

TABLE III. Complete Submission History

NOTEBOOK / CELL	SUBMISSION FILE	MAE	KEY CHANGE
Raw Cell 19	submission_deep_duo_gpu.csv	1.0952	LightGBM 74% + XGBoost 26% blend; evap_stress; windows [7,14,28,49,84]
Raw Cell 31	submission_nn_only_ram_friendly.csv	1.1040	GRU with attention; seq=90; multi-head attention; windows [7,14,28,49,84]
Raw Cell 32	submission_hybrid_cnn_gru.csv	0.9787	DroughtHybrid NN; seq=90; cosine LR; cyclical time encoding
Final Cell 7 (Raw Cell 40)	submission_70_window_region.csv	0.8582	Clean LightGBM; evap_stress; normals; windows [7,14,28,49,84]; region_avg_score only; no future-month
Final Cell 8 (Raw Cell 41)	submission_70_window_region_monthly.csv	0.8526	+ month_region_score prior; + future-month embedding; roll_std w $\geq$ 28; $\theta=0.20$
Final Cell 9 (Raw Cell 42)	...down_20_up_10.csv	—	Dual-threshold rounding only (floor $\theta=0.20$, round $\theta=0.10$); no model change
Final Cell 10 (Raw Cell 43)	...last_5_week...monthly.csv	0.8298	Expanded weather_cols to 8 vars; windows [7,14,21,28,35]; $\theta=0.20$
Final Cell 11 (Raw	...last_5_week...monthly.csv	0.8085	n_est=5000; LR=0.01; L1=2; L2=20; subsample=0.6

Cells 44–47)			; roll_std w $\geq$ 28; $\theta=0.25$
Final Cell 12 (Raw Cell 49)	...last_4&8_weeks...monthly.csv	0.78 96	+ evap_demand; windows [7,14,21,28,42] ; min_periods=1 ; $\theta=0.20$ — BEST SELECTED
Final Cell 14 (Raw Cell 50)	...10_rounded.csv	0.81 59	n_est=6000; subsample=0.7 ; $\theta=0.10$ + zero-snap below 0.25 — WORSE, not selected

C. Ablation Analysis

Table IV quantifies the contribution of each major design decision from paired submission comparisons.

TABLE IV. Feature and Design Ablation

DESIGN CHANGE	MAE Δ	COMPARISON
LightGBM single model vs. ensemble blend	−0.037	submission_weighted (1.0952) → Cell 7 (0.8582) combined
+ month_region_score + future-month embedding	−0.056	Cell 7 (0.8582) → Cell 8 (0.8526), continued gains in Cell 10
Windows [7,14,28,49,84] → [7,14,21,28,35] + 8-var weather_cols	−0.023	Cell 8 (0.8526) → Cell 10 (0.8298)
n_est=5000; L1=2; L2=20; subsample=0.6; $\theta=0.25$	−0.021	Cell 10 (0.8298) → Cell 11 (0.8085)
+ evap_demand; window 35→42; min_periods=1; $\theta=0.20$	−0.019	Cell 11 (0.8085) → Cell 12 (0.7896) — BEST
n_est=6000; zero-snap heuristic ($\theta=0.10$ + floor<0.25)	+0.026 (worse)	Cell 11 (0.8085) → Cell 14 (0.8159)
Transformer (seq=90) vs. LightGBM final	+0.314 (worse)	Cell 31 (1.1040) vs. Cell 12 (0.7896)
CNN-GRU (seq=90) vs. LightGBM final	+0.189 (worse)	Cell 32 (0.9787) vs. Cell 12 (0.7896)

D. Hyperparameter Sensitivity

Key findings from sequential submission comparisons:

- n_estimators 1000→5000 (lr=0.01): consistent improvement. Low learning rate requires many rounds to converge given the high feature dimensionality. Increasing further to 6000 (Cell 14) with

subsample=0.7 degraded performance, indicating overfitting onset.

- num_leaves 95→63: improved generalisation with the expanded 9-variable feature set. Fewer leaves reduce the model's ability to memorise region-specific noise.
- lambda_l2=20: strong L2 regularisation was necessary after adding evap_demand and widening the window set. Weaker regularisation (Cell 10, effectively L2≈0) underperformed by 0.040 MAE.
- Rounding threshold $\theta=0.20$ vs $\theta=0.25$: $\theta=0.20$ outperformed $\theta=0.25$ in the final configuration (Cell 12 vs Cell 11), suggesting that predictions in the range [0.20, 0.25] from the nearest integer contain genuine fractional signal and should not be snapped.
- Zero-snap heuristic (Cell 14): forcing predictions below 0.25 to exactly 0.0 degraded MAE by 0.026, confirming that near-zero scores carry real predictive signal rather than noise.
- min_periods=1: eliminates NaN rows at the start of each region's rolling window sequence, increasing effective training data and ensuring complete test coverage. Contributed to Cell 12's improvement over Cell 11.

VII. DISCUSSION

The development trajectory across both notebooks reveals a clear hierarchy of contribution. The single largest gain came from replacing ensemble blending with a properly engineered single LightGBM model incorporating climate normal baselines and region identity (−0.09 MAE). The addition of month_region_score and the future-month seasonal embedding contributed the next largest gain (−0.056 MAE combined with window refinement), confirming that drought risk is strongly seasonal and that forward-looking seasonal priors are more informative than backward-looking rolling statistics alone for long-horizon prediction.

The future-month embedding deserves specific emphasis. Rolling window features are entirely backward-looking: they summarise the past w days. The future-month embedding, by contrast, provides a direct forward-looking seasonal prior: for a prediction targeting week k , the model receives the historical average drought risk for the calendar month in which week k will fall. For horizons w_3 – w_5 where the forecast crosses a calendar month boundary, this dramatically improves accuracy by enabling the model to anticipate seasonal transitions rather than extrapolating recent trends.

The evap_demand index ($\text{tmp_max} / (\text{prec} \times 0.01)$) complements evap_stress ($\text{tmp_max} \times (100 - \text{humidity})$) by capturing a different dimension of drought forcing: the precipitation-to-temperature moisture balance rather than the heat-humidity dryness stress. Together they encode two mechanistically distinct aspects of atmospheric drought conditions, explaining why their joint inclusion outperforms either feature alone.

Deep learning architectures (Transformer, CNN-GRU) uniformly underperformed LightGBM, consistent with [7]. Beyond the general tabular data advantage of GBDT, the sequential models faced practical constraints: Kaggle GPU memory limits capped sequence length at 90 and batch sizes at 64–128, restricting effective model capacity. The tabular

LightGBM approach is inherently immune to these constraints, operating directly on the engineered feature vector without sequence overhead.

A key limitation is the absence of held-out time-series cross-validation. All model selection was performed using Kaggle public leaderboard feedback (public split only), which carries a risk of overfitting to the public test distribution. Future work should implement walk-forward cross-validation to ensure reliable offline model selection and reduce submission-count dependence. Additional directions include stacking diverse LightGBM configurations, incorporating soil moisture indices or NDVI as additional drought proxies, and applying Bayesian optimization for joint feature-hyperparameter search.

VIII. CONCLUSION

This paper documented the complete iterative development of a drought risk score forecasting system across two Kaggle notebooks spanning 52 and 15 cells, respectively. The final LightGBM multi-output regression pipeline combines two composite drought indices (evap_stress and evap_demand), region-month climate normal baselines, multi-scale rolling window statistics with min_periods=1 over windows [7, 14, 21, 28, 42] days, regional anomaly features, historical and monthly region score priors, a calendar-aware future-month seasonal embedding, and adaptive rounding post-processing ($\theta = 0.20$). The selected best submission achieves a public leaderboard MAE of 0.7896, a 28.0% improvement over the initial weighted ensemble baseline (MAE 1.0952), and is substantially better than all evaluated deep learning baselines. The complete development history is documented in both notebooks and all results are reproducible

from the GitHub repository following the README.md instructions.

IX. ACKNOWLEDGMENT

The authors thank the Data Mining course instructors for the dataset and competition framework. All experiments were conducted on Kaggle Notebooks using GPU resources provided by Kaggle.

X. REFERENCES

- [1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794.
- [2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 3146–3154.
- [3] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, Nov. 2011.
- [5] T. B. McKee, N. J. Doesken, and J. Kleist, "The relationship of drought frequency and duration to time scales," in Proc. 8th Conf. Applied Climatology, Anaheim, CA, USA, Jan. 1993, pp. 179–183.
- [6] A. K. Mishra and V. P. Singh, "A review of drought concepts," J. Hydrology, vol. 391, no. 1–2, pp. 202–216, Sep. 2010.
- [7] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "'Why do tree-based models still outperform deep learning on typical tabular data?'" NeurIPS 2022, pp. 507–520.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 5998–6008.